Lecture 10: The Jacobian Matrix & Singularities

Iñigo Iturrate

Associate Professor

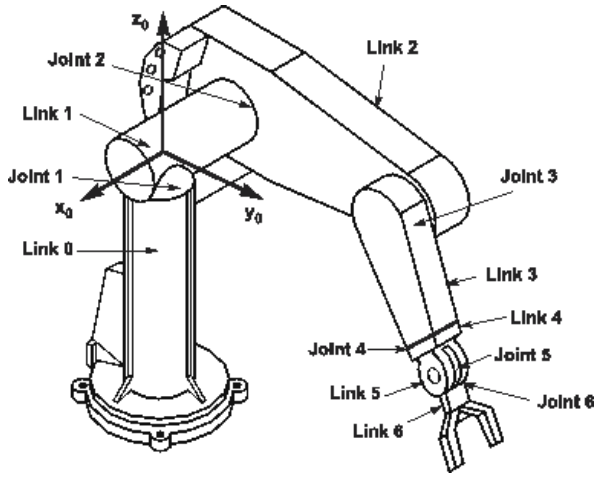
SDU Robotics,

The Maersk McKinney Moller Institute,

University of Southern Denmark

Ø27-604-3

inju@mmmi.sdu.dk



SDU ROBOTICS

What is Forward Kinematics? What do we use it for?

Nobody has responded yet.

Hang tight! Responses are coming in.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What is Inverse Kinematics? What do we use it for? What problems do we associate with it?

Nobody has responded yet.

Hang tight! Responses are coming in.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What are we doing today?

1. Introduction to Robotics & Recap of Linear Algebra and Mathematical Notation
2. Translations & Rotation Matrices
3. Other Representations for Orientation
4. Transformation Matrices
5. Forward Kinematics & DH Parameters
6. Inverse Kinematics

7. Kinematic Simulation
8. Schematic Representations of Robots, Modeling a Real Robot

9. Angular Velocities, Velocity Propagation & Trajectory Generation
10. **The Jacobian Matrix, Singularities** ***(Today)***
11. Analytical & Geometrical Jacobian, Numerical IK, Statics
12. Manipulability, More on the Robotic Systems Toolbox



"Robots that stay still",
i.e. positions/orientations

Simulation

"Robots that move",
i.e. velocities

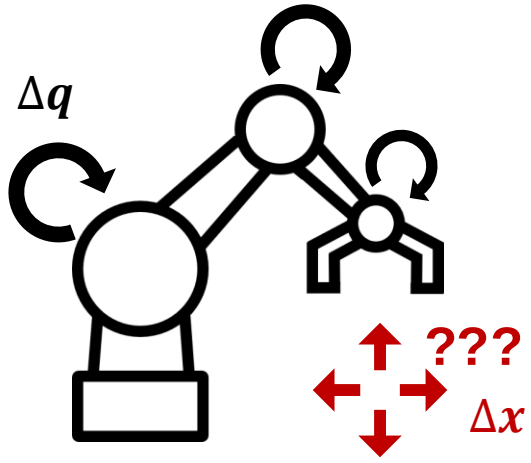
SDU ROBOTICS

Today…

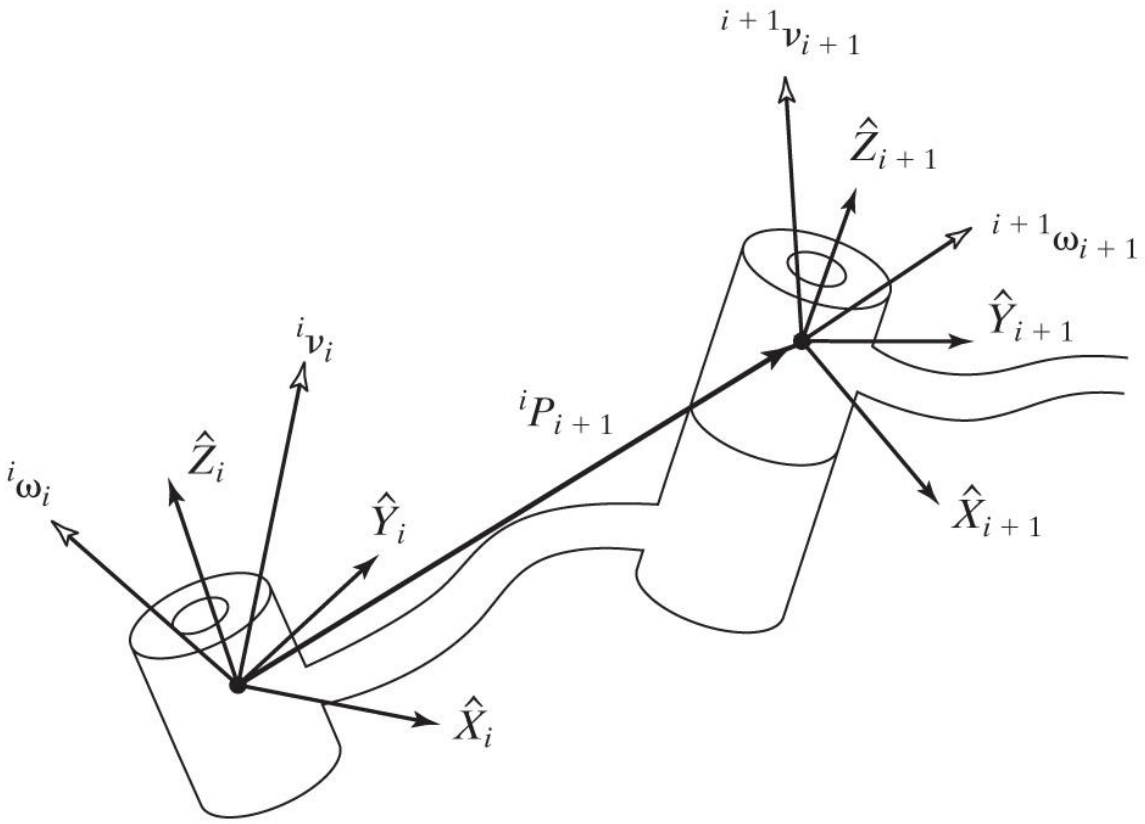
So far, we have worked with **robots at standstill** and described their **positions**,
both in **joint** and **Cartesian space**.

Often, however, we will want to consider **velocities** when **describing motion**.

Particularly, we might want to see how a **small change of joint angles** influences the **change in end-effector position**, and vice versa.



Recap: Velocity of Links – Revolute Joints



1. Total angular velocity seen from frame i:

$$^{i}\omega_{i+1} = {}^{i}\omega_i + {}^{i}_{i+1}R\dot{\theta}_{i+1}\,{}^{i+1}\hat{Z}_{i+1}$$

Rotation of joint i + previous joints

Rotation of joint $i+1$

2. Total linear velocity seen from frame i:

$$^{i}v_{i+1} = {}^{i}v_i + {}^{i}\omega_i \times {}^{i}P_{i+1}$$

Linear motion of joint i + previous joints

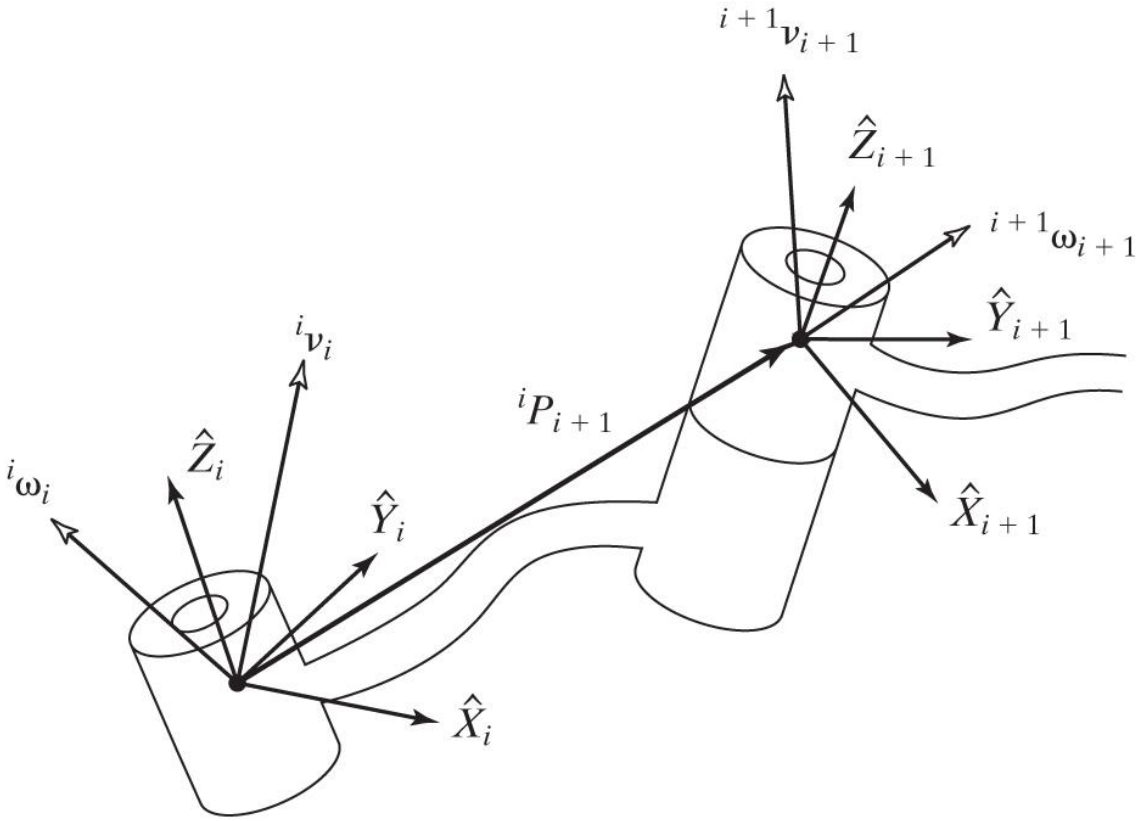
Rotation of joint i + previous joints

3. We can convert from frame i to $i+1$ with a rotation matrix:

$$^{i+1}_{i}R\ {}^{i}\omega_{i+1} = {}^{i+1}_{i}R\left({}^{i}\omega_i + {}^{i}_{i+1}R\dot{\theta}_{i+1}\,{}^{i+1}\hat{Z}_{i+1}\right)$$

$$^{i+1}_{i}R\ {}^{i}v_{i+1} = {}^{i+1}_{i}R\left({}^{i}v_i + {}^{i}\omega_i \times {}^{i}P_{i+1}\right)$$

Recap: Velocity of Links – Prismatic Joints



The exact same logic follows for a **prismatic joint**:

$$^{i+1}\omega_{i+1} = {}^{i+1}_{i}R\ {}^{i}\omega_{i}$$

$$^{i+1}v_{i+1} = {}^{i+1}_{i}R(\ {}^{i}v_{i} + {}^{i}\omega_{i} \times {}^{i}P_{i+1}) + \dot{d}_{i+1}\,{}^{i+1}\hat{Z}_{i+1}$$

SDU ROBOTICS

Topics for Today

Part I: The Jacobian Matrix

- Math background: partial derivatives
- Geometric and Analytical Jacobian
- Building a Geometric Jacobian
- Forward Velocity Kinematics

Part II: Singularities

- Inverse Velocity Kinematics
- Singularities as seen:
  - From the Jacobian
  - From Analytical (closed-form) solutions to IK

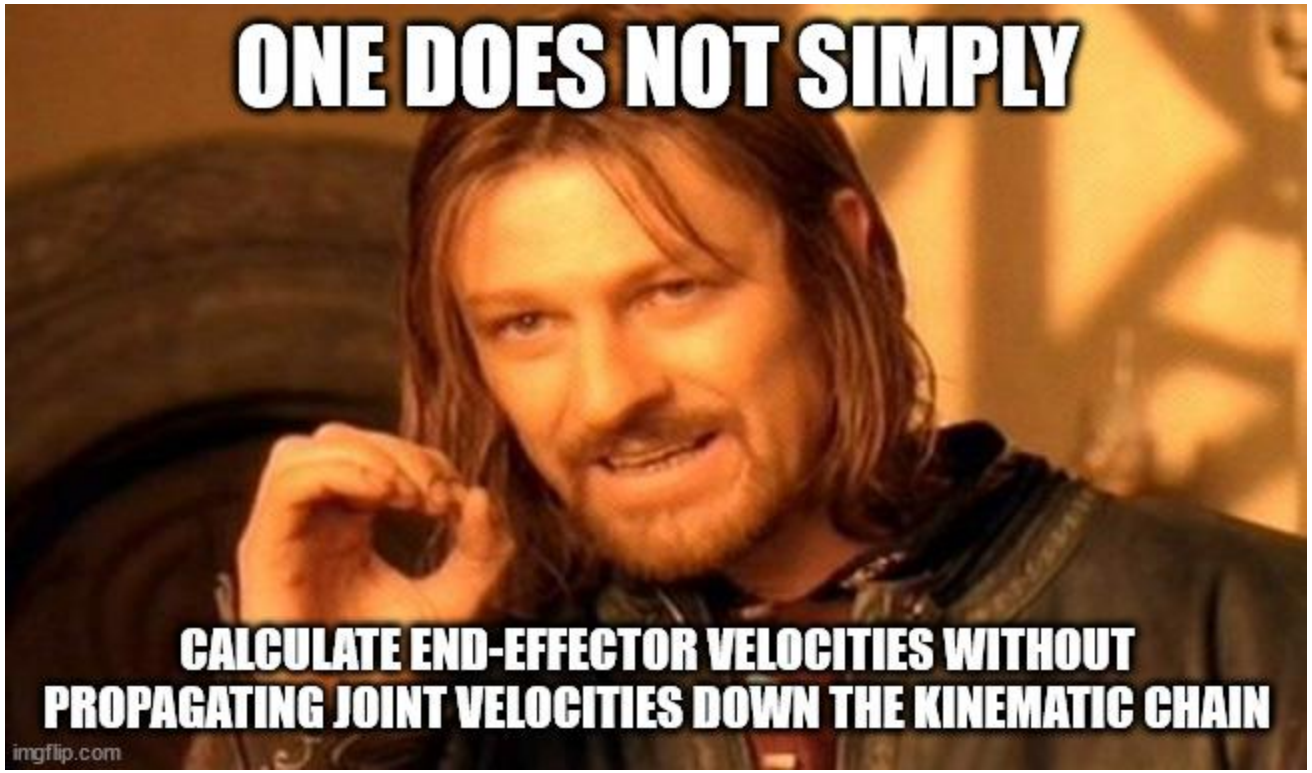
Part I: The Jacobian Matrix

What are we building up to?

We now have **expressions for** the **linear and angular velocities** of a **link** depending on its **joint motion**.

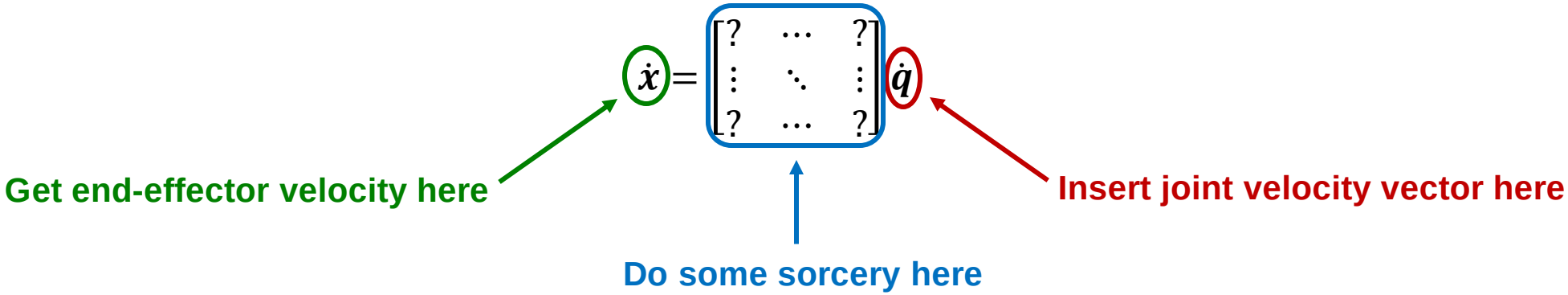
We want to **calculate how joint velocities affect the end-effector velocity**.

Problem: **link velocities** (caused by joint velocities) **contribute to each other** and **propagate** down the chain.



What do we want?

Ideally:



(This sorcery is called the Jacobian Matrix)

Before we get there… a few math concepts

In your math courses, you have mostly worked with **univariate functions** of the form $f(x)$, e.g.

$$f(x) = x^2 - 3x + 5$$

In real life, most functions are **multivariate**, i.e. they **depend on multiple variables** and have form $f(x_1, x_2, \cdots)$, e.g.

$$f(x, y) = yx^2 - 3xy + 5y - 2$$

The functions describing **robot kinematics** are **multivariate** as:

$$\boldsymbol{q}, \dot{\boldsymbol{q}} \in \mathbb{R}^n, \qquad \boldsymbol{x}, \dot{\boldsymbol{x}} \in \mathbb{R}^6$$

This means we need to redefine the concept of **derivative for multivariate functions:**

https://web.mit.edu/jorloff/www/jmoapplets/ddm-mathinsight/ddm-jmo.html

Partial Derivatives: Chain Rule

If we have a function $\boldsymbol{u = u(x, y)}$, then the **differential of $\boldsymbol{u}$** can be written as:

$$\delta u = \frac{\partial u}{\partial x}\delta x + \frac{\partial u}{\partial y}\delta y$$

But let us say that **each of the variables is a function of a *single* other variable**, t, such that $\boldsymbol{x = x(t)}$ and $\boldsymbol{y = y(t)}$. We then end up with $u = u(x(t), y(t))$.

If we **take the limits $\boldsymbol{\delta x \to 0}$, $\boldsymbol{\delta y \to 0}$ and $\boldsymbol{\delta t \to 0}$**, we end up with:

$$\frac{du}{dt} = \frac{\partial u}{\partial x}\frac{dx}{dt} + \frac{\partial u}{\partial y}\frac{dy}{dt}$$

The Jacobian Matrix

The **Jacobian matrix** is a **multidimensional form of the derivative**. It is a **matrix of partial derivatives**.

If we have, e.g. 6 functions of 6 variables each:

$$\begin{aligned} y_1 &= f_1(x_1, x_2, x_3, x_4, x_5, x_6) \\ y_2 &= f_2(x_1, x_2, x_3, x_4, x_5, x_6) \\ &\vdots \\ y_6 &= f_6(x_1, x_2, x_3, x_4, x_5, x_6) \end{aligned}$$

Which we can re-write: $\boldsymbol{y = F(x)}$

And we want to **calculate the differentials** of each function based on the input:

$$\begin{aligned} \delta y_1 &= \frac{\partial f_1}{\partial x_1}\delta x_1 + \frac{\partial f_1}{\partial x_2}\delta x_2 + \cdots + \frac{\partial f_1}{\partial x_6}\delta x_6 \\ \delta y_2 &= \frac{\partial f_2}{\partial x_1}\delta x_1 + \frac{\partial f_2}{\partial x_2}\delta x_2 + \cdots + \frac{\partial f_2}{\partial x_6}\delta x_6 \\ &\vdots \\ \delta y_6 &= \frac{\partial f_6}{\partial x_1}\delta x_1 + \frac{\partial f_6}{\partial x_2}\delta x_2 + \cdots + \frac{\partial f_6}{\partial x_6}\delta x_6 \end{aligned}$$

We can re-write the whole thing as: $\boldsymbol{\delta y = J(x)\delta x}$

$$\boldsymbol{J(x)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_1}{\partial x_2}(\boldsymbol{x}) & \cdots & \frac{\partial f_1}{\partial x_6}(\boldsymbol{x}) \\ \frac{\partial f_2}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_2}{\partial x_2}(\boldsymbol{x}) & \cdots & \frac{\partial f_2}{\partial x_6}(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_6}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_6}{\partial x_2}(\boldsymbol{x}) & \cdots & \frac{\partial f_6}{\partial x_6}(\boldsymbol{x}) \end{bmatrix}$$

SDU ROBOTICS

The Forward Velocity Kinematics Equation

Now let us put this in the **context of our robotics problem**:

From the **definition** of Jacobian:

$$\frac{\partial \boldsymbol{x}}{\partial \boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q})$$

By applying the **chain rule**:

$$\frac{\partial \boldsymbol{x}}{\partial \boldsymbol{q}} \frac{d\boldsymbol{q}}{dt} = \boldsymbol{J}(\boldsymbol{q}) \frac{d\boldsymbol{q}}{dt}$$

Re-arranging:

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{J}(\boldsymbol{q}) \frac{d\boldsymbol{q}}{dt}$$

$$\boxed{\dot{\boldsymbol{x}} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}}$$



The Two Jacobians in Robotics

Remember that we can **describe Cartesian-space velocity** in (at least) **two ways**, depending on how we describe **rotational velocity**:

- Using **angular** velocity: $\dot{x} = \begin{bmatrix} \dot{p} \\ \omega \end{bmatrix}$
- Using the **derivative of Euler angles**: $\dot{x} = v = \begin{bmatrix} \dot{p} \\ \dot{\phi} \end{bmatrix}$

We can split the Jacobian in two: $J = \begin{bmatrix} J_P \\ J_O \end{bmatrix}$

Jacobian for Position *(This stays the same)*

Jacobian for Orientation

J_O based on angular velocity

J_O based on derivative of Euler angles

Geometric Jacobian

We will only study this

Analytical Jacobian

Building a Geometric Jacobian

We first **split the matrix** into (3×1) column vectors, $\boldsymbol{J}_{Pi}$ and $\boldsymbol{J}_{Oi}$, where each element i represents the **contribution of a single joint q_i to either the position or orientation**:

$$\boldsymbol{J} = \begin{bmatrix} \boldsymbol{J}_{P1} & \cdots & \boldsymbol{J}_{Pn} \\ \boldsymbol{J}_{O1} & & \boldsymbol{J}_{On} \end{bmatrix}$$

Then, for each joint i:

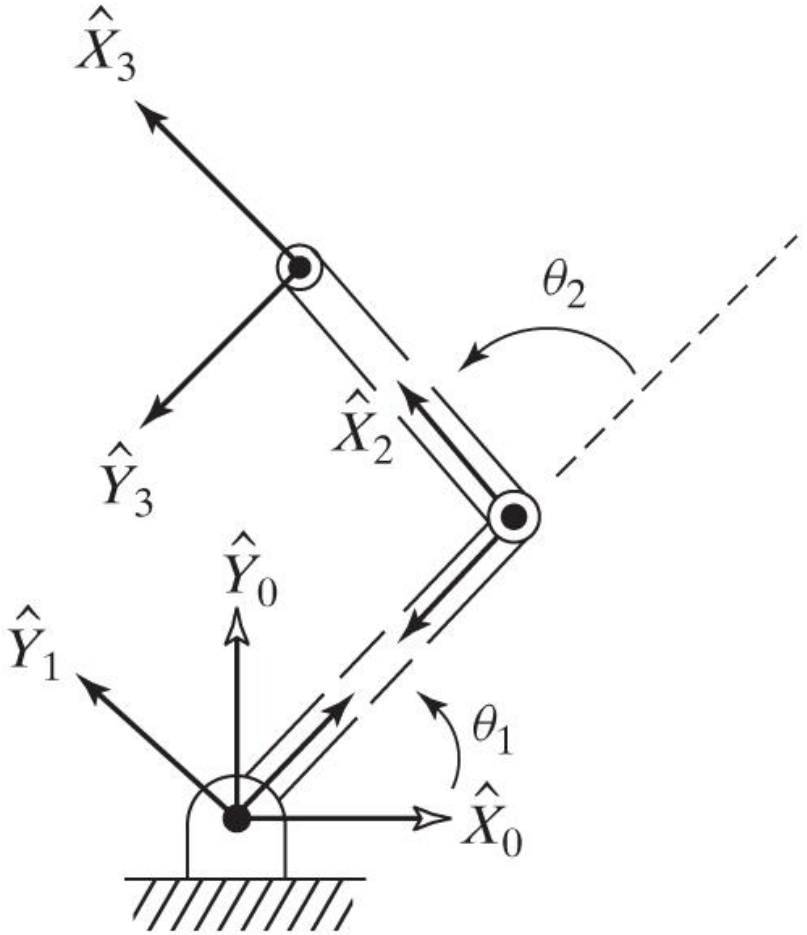
- **Revolute**: $\boldsymbol{J}_{Pi} = {}^0\hat{Z}_i \times ({}^0P_e - {}^0P_i)$, $\boldsymbol{J}_{Oi} = {}^0\hat{Z}_i$
- **Prismatic:** $\boldsymbol{J}_{Pi} = {}^0\hat{Z}_i$, $\boldsymbol{J}_{Oi} = 0$

$${}^0\hat{Z}_i = {}^0_iR \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

${}^0P_e$ is the position of the end-effector

${}^0P_i$ is the position of the origin of frame {i}

Jacobian for 2R Manipulator – Exercise (15-20 minutes)



Consider the 2-link planar robot with the frame-assignment in the figure (from Modified DH Parameters).

1. Download the file *FK_2_link_planar_sym.m* from *istlearning.*
2. Follow the procedure in the previous slide to calculate a (2×2) Jacobian for the manipulator

Calculating the Positional Jacobian Analytically

The **positional part** of the Jacobian, $\boldsymbol{J}_{Pi}$, can instead be **calculated analytically**.

Given the **forward kinematics** transformation matrix:

$$ {}^{0}_{e}\boldsymbol{T}(\boldsymbol{\theta}) = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Take the first three rows of the last column:

$$ {}^{0}_{e}\boldsymbol{P}(\boldsymbol{\theta}) = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \rightarrow \boldsymbol{J}_P = \frac{\partial \boldsymbol{P}}{\partial \boldsymbol{q}} = \begin{bmatrix} \frac{\partial p_x}{\partial q_1} & \cdots & \frac{\partial p_x}{\partial q_1} \\ \frac{\partial p_y}{\partial q_1} & \cdots & \frac{\partial p_y}{\partial q_1} \\ \frac{\partial p_z}{\partial q_1} & \cdots & \frac{\partial p_z}{\partial q_1} \end{bmatrix} $$

Do NOT do this by hand!

If you have the FK in symbolic MATLAB, call:

```
J = jacobian(P)
```

https://se.mathworks.com/help/symbolic/sym.jacobian.html

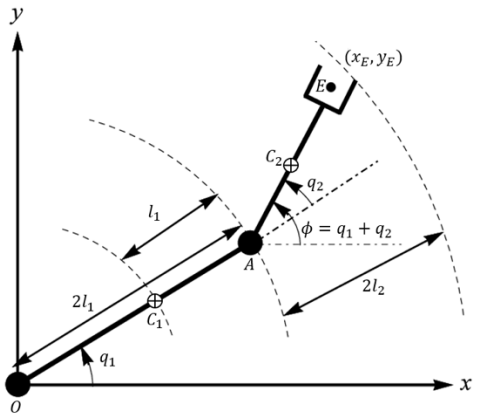
Dimensions of Jacobian – Discuss Aloud (10 minutes)

What are the dimensions of the Jacobian matrices for the following robots?

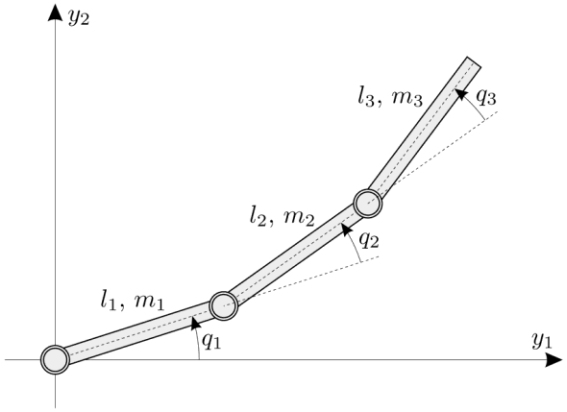
Consider:

1. *What are the dimensions of the input (joint) space?*
2. *What are the dimensions of the output (Cartesian) space?*

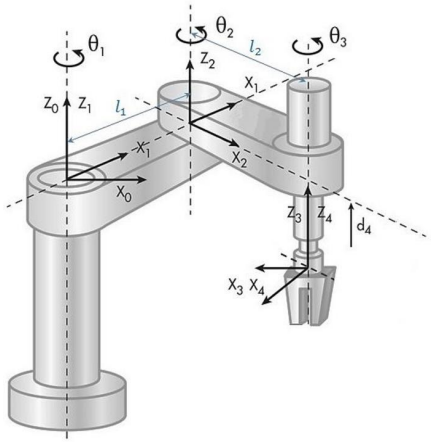
2R Planar



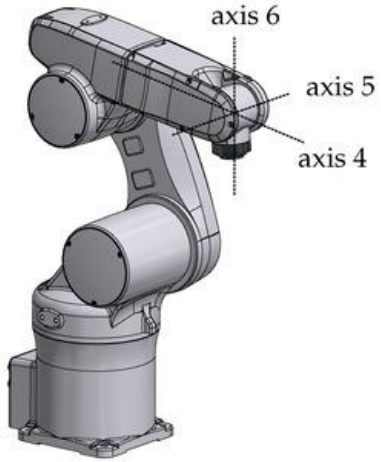
3R Planar



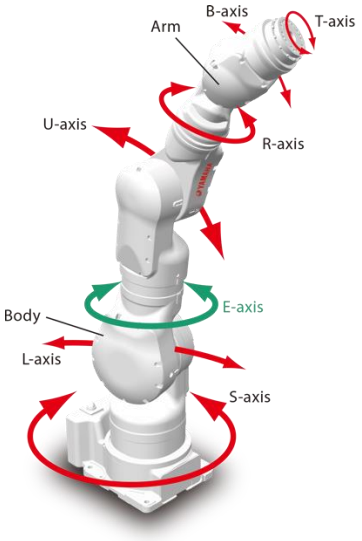
SCARA



6R



7R



Part II: Singularities

Forward and Inverse Velocity Kinematics

Given the **forward velocity kinematics** equation, and **assuming** that the **inverse of the Jacobian exists**, we can obtain the inverse velocity kinematics equation:

$$\dot{\boldsymbol{x}} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}} \quad \longleftrightarrow \quad \dot{\boldsymbol{q}} = \boldsymbol{J}^{-1}(\boldsymbol{q})\dot{\boldsymbol{x}}$$

This is a **very important result in robotics**.

The forward/inverse velocity kinematics equations are the **basis of numerical IK methods**.

The Problem with the Jacobian Inverse

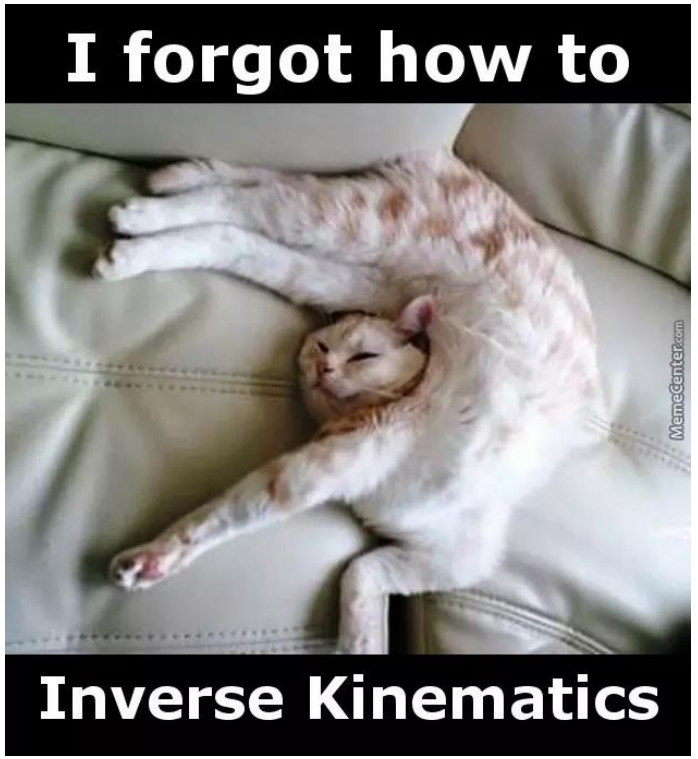
Consider **the inverse velocity kinematics equation**:

$$\dot{q} = J^{-1}(q)\dot{x}$$

Not all matrices have an inverse…
This does not always exist!

Do you see any potential issues here?

So… what happens then?



Singularities: Non-existence of inverse

When does the inverse of a matrix not exist?

Due to a **non-square** matrix:

- Example: **a redundant robot** (e.g. 7-DOF in a 3D space)

$$\dot{\boldsymbol{x}} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$$

$$\begin{bmatrix} \dot{\boldsymbol{p}} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} \boldsymbol{J}_{P1} & \cdots & \boldsymbol{J}_{P7} \\ \boldsymbol{J}_{O1} & & \boldsymbol{J}_{O7} \end{bmatrix} \begin{bmatrix} q_1 \\ \vdots \\ q_7 \end{bmatrix}$$

$(6 \times 1) \qquad (6 \times 7) \qquad (7 \times 1)$



Due to a **rank-defficient square** matrix

Rank & Determinant of the Jacobian

Remember: The **rank of a matrix** is the **number of linearly independent columns**.

Full rank: The rank of the matrix is the maximum possible for a matrix of that size.

Rank-defficient/singular: The matrix is not full rank.

- The **robot cannot move in one or more directions**.
- At a **singular configuration**, the **determinant** of the Jacobian will be **zero** (and close to zero when close to a singularity).

Discuss:
Think about this last statement in connection with:

- The definition of the inverse of a matrix: $\boldsymbol{J^{-1}} = \frac{1}{\det(\boldsymbol{J})}(\mathrm{adj}(\boldsymbol{J}))$
- The inverse velocity kinematics equation: $\dot{\boldsymbol{q}} = \boldsymbol{J}^{-1}(\boldsymbol{q})\dot{\boldsymbol{x}}$

What can we conclude about the joint velocities of a robot when moving close to a singularity?

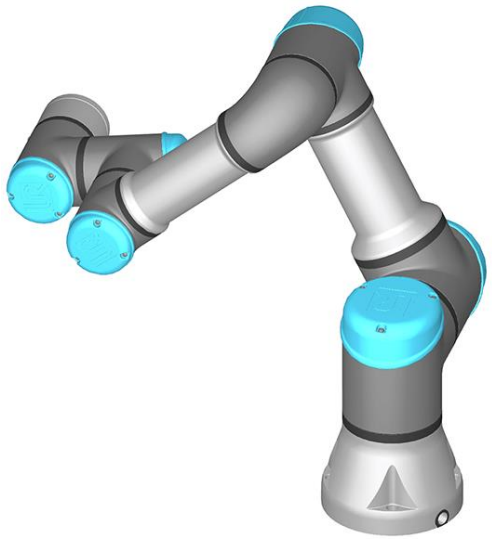
Singularities – Example: UR Robot



Singularities of UR: Relating them to IK Solutions

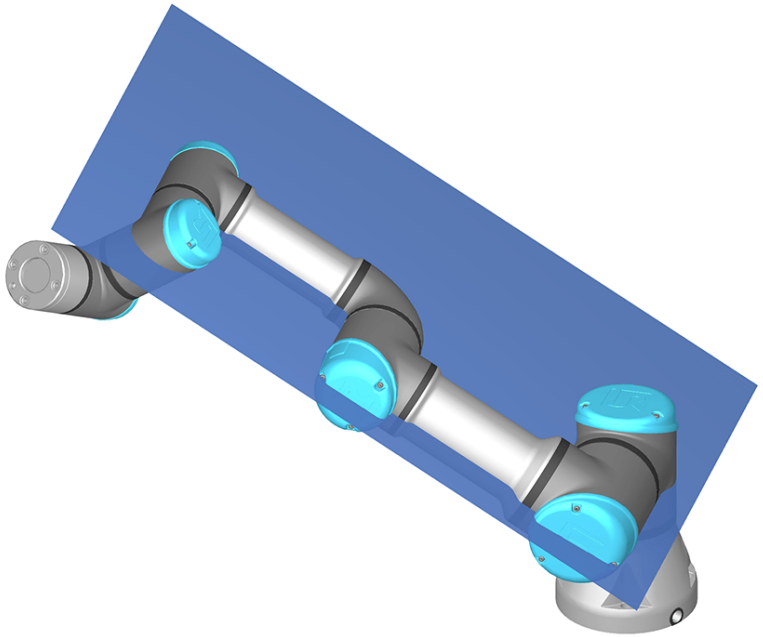
Wrist singularity

- Axes of joints 4 and 6 parallel
- $\theta_5 = 0°$, $\theta_5 = \pm 180°$ or $\theta_5 = \pm 360°$



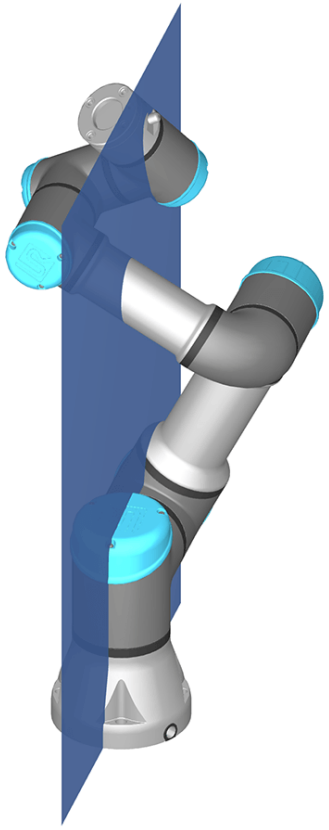
Elbow singularity

- Axes of joints 2, 3 and 4 coplanar
- $\theta_3 = 0°$



Shoulder singularity

- Intersection of axes of joints 5 and 6 coplanar with axes of joints 1 and 2.



SDU ROBOTICS

Singularities – Example: ABB Robot



What if the Jacobian is NOT square?

What do we do with robots whose Jacobians are not square, e.g. redundant robots?

- Jacobian **Transpose**: $\dot{\boldsymbol{q}} = \alpha \boldsymbol{J}^T \dot{\boldsymbol{x}}$ (for some small $\alpha > 0$)
  - This can behave ”close enough” to the inverse assuming a small enough α.

- Jacobian (Moore-Penrose) **Pseudoinverse**: $\dot{\boldsymbol{q}} = \boldsymbol{J}^{\dagger} \dot{\boldsymbol{x}}$ (where † is the pseudoinverse)
  - This is a generalization of the concept of inverse for non-square matrices.

…which, in MATLAB, can be calculated using `pinv()`:
https://se.mathworks.com/help/matlab/ref/pinv.html?s_tid=doc_ta

Practical Applications of the Jacobian

The Jacobian is **used everywhere** in robotics:

- **Numerical Inverse Kinematics**
- **Manipulability Metrics** (checking how ”free” the robot is to move/apply a force in each direction)
- **Statics** (Seeing how forces at the end-effector propagate to the individual links)

We will discuss this in the remaining lectures

- **Dynamics:**
  - Force Propagation
- **Robot Control**
  - Position Control
  - Velocity Control
  - Force Control

You will study this in other courses

SDU ROBOTICS

Recap: What have we discussed today

- **Partial derivatives** express how a **multivariate function** reacts to **changes in each of the input variables**.
  - The **Jacobian** is a **matrix form** of the **partial derivative**
- In **robotics**, we use the Jacobian to **relate joint-space velocities to Cartesian space velocities.**
- The **inverse of the Jacobian cannot be calculated** if the robot is **in a singular configuration**.
  - In this case, the **robot cannot move in certain directions**.
  - **Joint velocities** close to singularities will be **very large**.

Take home message:

The **Jacobian** relates, through a matrix of partial derivatives, **how the velocity of each joint affects the velocity of the end-effector**.

Moving close to **singularities** (where the **Jacobian cannot be inverted**) results in **high joint velocities**, which has important practical consequences.

SDU ROBOTICS

Thank you for today.

Iñigo Iturrate

Ø27-604-3

inju@mmmi.sdu.dk